

**LAPORAN PRAKTIKUM
ALGORITMA & STUKTUR DATA**

Sorting Algorithm



Di Susun Oleh :

Nama : M. SYARFANI
NIM : 2025573010015
Kelas : TI. 1C
Mata Kuliah : ALGORITMA & STUKTUR DATA
Program Studi : TEKNIK INFORMATIKA
Dosen : Muhammad Reza Zulman, S.ST.,M.Sc

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
TAHUN 2026/2027

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT yang telah memberikan rahmat dan karunia-Nya sehingga saya dapat menyelesaikan Laporan Praktikum Algoritma dan Struktur Data Praktikum 9 ini dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban atas pelaksanaan praktikum yang telah dilakukan. Pada praktikum ini dibahas beberapa materi penting dalam algoritma dan struktur data, yaitu Memoization, Merge Sort, Quick Sort, Sorting $O(n^2)$, dan Counting Sort. Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep, implementasi, serta analisis kinerja berbagai algoritma yang digunakan dalam pengolahan data. Dalam penyusunan laporan ini, saya memperoleh banyak bantuan, arahan, dan bimbingan dari berbagai pihak. Oleh karena itu, saya mengucapkan terima kasih kepada Bapak Muhammad Reza Zulman, S.ST., M.Sc. selaku dosen pengampu mata kuliah Algoritma dan Struktur Data yang telah memberikan ilmu, arahan, serta bimbingan selama proses perkuliahan dan praktikum berlangsung. Saya menyadari bahwa laporan ini masih memiliki kekurangan dan jauh dari kata sempurna. Oleh karena itu, saya mengharapkan kritik dan saran yang membangun demi perbaikan dan penyempurnaan laporan di masa yang akan datang. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan menjadi referensi dalam mempelajari materi Algoritma dan Struktur Data.

Buket Rata 17 Juni 2026

M. Syarfani

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	4
1.1 Latar Belakang.....	4
1.2 Tujuan Praktikum	4
1.3 Manfaat Praktikum	5
BAB II PEMBAHASAN.....	6
2.1 Memoization.....	6
2.2 Merge Sort	6
2.3 Quick Sort.....	7
2.4 Sorting $O(n^2)$	7
2.5 Bubble Sort.....	8
BAB III PRAKTIKUM	9
3.1 Langkah Langkah Praktikum Sorting Algorithm ,	9
BAB IV PENUTUP	16
4.1 Kesimpulan.....	16
DAFTAR PUSTAKA	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Algoritma pengurutan (sorting) merupakan salah satu konsep penting dalam ilmu komputer dan pengolahan data. Pengurutan data bertujuan untuk menyusun data dalam urutan tertentu sehingga lebih mudah dicari, dianalisis, dan dikelola. Dalam pengembangan perangkat lunak, penggunaan algoritma yang efisien sangat diperlukan untuk meningkatkan kinerja sistem, terutama ketika menangani data dalam jumlah besar. Selain algoritma pengurutan, teknik optimasi seperti Memoization juga memiliki peran penting dalam meningkatkan efisiensi program. Memoization memungkinkan hasil perhitungan yang telah dilakukan sebelumnya disimpan sementara sehingga tidak perlu dihitung ulang ketika dibutuhkan kembali. Teknik ini dapat mengurangi waktu eksekusi program secara signifikan. Pada Praktikum 9 ini dipelajari beberapa algoritma dan teknik penting, yaitu Memoization, Merge Sort, Quick Sort, Sorting $O(n^2)$, dan Counting Sort. Setiap algoritma memiliki karakteristik, kelebihan, dan kompleksitas waktu yang berbeda. Dengan mempelajari dan mengimplementasikan algoritma tersebut, mahasiswa dapat memahami cara memilih algoritma yang tepat sesuai kebutuhan serta menganalisis performa masing-masing algoritma dalam menyelesaikan suatu permasalahan.

1.2 Tujuan Praktikum

Adapun tujuan dari pelaksanaan Praktikum 9 ini adalah sebagai berikut:

- Memahami konsep Memoization sebagai teknik optimasi dalam pemrograman.
- Memahami cara kerja algoritma Merge Sort.
- Memahami cara kerja algoritma Quick Sort.
- Memahami algoritma Sorting $O(n^2)$ seperti Bubble Sort, Selection Sort, dan Insertion Sort.
- Memahami konsep dan implementasi Counting Sort.
- Menganalisis efisiensi dan kompleksitas waktu dari berbagai algoritma pengurutan.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari pelaksanaan praktikum ini adalah :

- Menambah pemahaman mengenai teknik optimasi dan algoritma pengurutan data.
- Meningkatkan kemampuan dalam mengimplementasikan algoritma menggunakan bahasa pemrograman JavaScript.
- Melatih kemampuan analisis terhadap efisiensi algoritma berdasarkan kompleksitas waktu.
- Membantu mahasiswa memahami perbedaan karakteristik berbagai algoritma sorting.
- Menjadi bekal dalam menyelesaikan permasalahan pengolahan data pada pengembangan perangkat lunak.

BAB II

PEMBAHASAN

2.1 Memoization

Memoization merupakan teknik optimasi yang digunakan dalam pemrograman untuk meningkatkan kinerja suatu algoritma dengan cara menyimpan hasil perhitungan yang telah dilakukan sebelumnya. Ketika fungsi dipanggil kembali dengan parameter yang sama, program tidak perlu melakukan perhitungan ulang, melainkan cukup mengambil hasil yang telah tersimpan. Teknik memoization sering digunakan pada algoritma rekursif yang memiliki banyak perhitungan berulang, seperti perhitungan deret Fibonacci. Dengan memoization, kompleksitas waktu algoritma dapat berkurang secara signifikan dibandingkan pendekatan rekursif biasa. Memoization merupakan teknik optimasi dalam pemrograman yang bermanfaat untuk meningkatkan efisiensi suatu algoritma dengan cara menyimpan hasil perhitungan yang telah dilakukan sebelumnya ke dalam memori sementara (cache).

2.2 Merge Sort

Merge Sort adalah algoritma pengurutan (sorting) yang menggunakan metode Divide and Conquer. Metode ini bekerja dengan cara membagi data menjadi beberapa bagian yang lebih kecil, kemudian mengurutkan masing-masing bagian, dan akhirnya menggabungkan kembali bagian-bagian tersebut menjadi data yang telah terurut. Proses kerja Merge Sort dimulai dengan membagi array menjadi dua bagian yang sama besar secara berulang hingga setiap bagian hanya memiliki satu elemen. Setelah itu, setiap bagian digabungkan kembali dengan membandingkan elemen-elemen yang ada sehingga menghasilkan urutan data yang benar. Karena proses pembagian dan penggabungan dilakukan secara rekursif, Merge Sort mampu menghasilkan pengurutan yang efisien meskipun jumlah data cukup besar. Salah satu keunggulan Merge Sort adalah memiliki kompleksitas waktu yang stabil, yaitu $O(n \log n)$ pada kondisi terbaik, rata-rata, maupun terburuk. Oleh karena itu, algoritma ini sering digunakan untuk mengurutkan data dalam jumlah besar. Namun, Merge Sort memiliki kelemahan yaitu membutuhkan memori tambahan untuk menyimpan hasil sementara selama proses penggabungan data.

2.3 Quick Sort

Quick Sort merupakan salah satu algoritma pengurutan (sorting) yang menggunakan metode Divide and Conquer. Algoritma ini bekerja dengan memilih satu elemen sebagai pivot, kemudian membagi data menjadi dua bagian, yaitu elemen yang nilainya lebih kecil dari pivot dan elemen yang nilainya lebih besar dari pivot. Setelah proses pembagian selesai, masing-masing bagian akan diurutkan kembali secara rekursif hingga seluruh data tersusun secara terurut. Proses kerja Quick Sort dimulai dengan menentukan sebuah pivot dari kumpulan data. Selanjutnya, seluruh elemen dibandingkan dengan pivot dan ditempatkan pada posisi yang sesuai. Elemen yang lebih kecil dari pivot ditempatkan di sebelah kiri, sedangkan elemen yang lebih besar ditempatkan di sebelah kanan. Setelah itu, proses yang sama dilakukan pada kedua bagian tersebut hingga tidak ada lagi data yang perlu dibagi. Quick Sort dikenal sebagai salah satu algoritma pengurutan yang sangat cepat dan efisien dalam praktik. Algoritma ini banyak digunakan karena mampu mengurutkan data dengan performa yang baik serta tidak membutuhkan memori tambahan yang besar. Namun, performa Quick Sort dapat menurun apabila pemilihan pivot kurang tepat, misalnya ketika data sudah hampir terurut sehingga menghasilkan pembagian yang tidak seimbang.

2.4 Sorting $O(n^2)$

Sorting $O(n^2)$ merupakan kelompok algoritma pengurutan yang memiliki kompleksitas waktu kuadrat. Artinya, waktu yang dibutuhkan untuk mengurutkan data akan meningkat secara signifikan seiring bertambahnya jumlah data yang diproses. Algoritma dalam kategori ini umumnya mudah dipahami dan diimplementasikan, sehingga sering digunakan untuk mempelajari konsep dasar pengurutan data. Pada praktikum ini, algoritma yang termasuk dalam kelompok Sorting $O(n^2)$ adalah Bubble Sort, Selection Sort, dan Insertion Sort. Ketiga algoritma tersebut memiliki cara kerja yang berbeda, namun memiliki kompleksitas waktu rata-rata yang sama, yaitu $O(n^2)$.

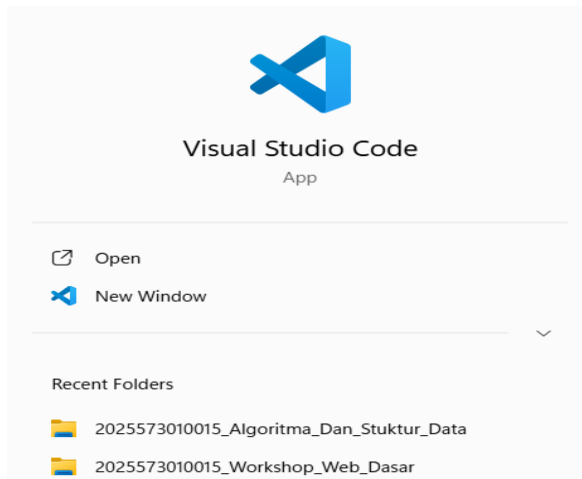
2.5 Bubble Sort

Bubble Sort merupakan salah satu algoritma pengurutan (sorting) yang paling sederhana dan mudah dipahami. Algoritma ini bekerja dengan cara membandingkan dua elemen yang berdekatan, kemudian menukarkan posisinya apabila urutannya tidak sesuai. Proses perbandingan dan pertukaran tersebut dilakukan secara berulang hingga seluruh data tersusun dalam urutan yang diinginkan. Pada setiap putaran, elemen dengan nilai terbesar akan bergerak ke posisi akhir array, sehingga proses ini menyerupai gelembung (bubble) yang naik ke permukaan air. Oleh karena itu, algoritma ini disebut Bubble Sort. Pengurutan akan terus dilakukan sampai tidak ada lagi elemen yang perlu ditukarkan. Bubble Sort memiliki kelebihan yaitu mudah dipahami dan mudah diimplementasikan, sehingga sering digunakan untuk mempelajari konsep dasar algoritma pengurutan. Namun, algoritma ini kurang efisien untuk data berukuran besar karena membutuhkan banyak proses perbandingan dan pertukaran data.

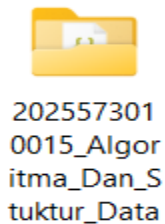
BAB III PRAKTIKUM

3.1 Langkah Langkah Praktikum Sorting Algorithm ,

1. Langkah Pertama adalah Membuka Visual Studio nya ,



2. Kemudian Pilih Folder yang sesuai dengan nama repository nya , Lalu select folder ,



3. Setelah itu , Buka Git Bash. Kemudian navigasikan ke folder repository GitHub kamu yang sudah di-clone.

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data (main)
$ cd 2025573010015_Algoritma_Dan_Stuktur_Data
```

4. Kemudian buat folder baru praktikum-9, untuk menyimpan semua file di dalam nya ,

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data (main)  
$ mkdir praktikum-9
```

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data (main)  
$ cd praktikum-9
```

5. Lalu , inisialisasi npm project ,

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-9 (main)  
$ npm init -y
```

6. Di dalam Praktikum-9 , Buatlah file baru Bernama 01-memoization.js ,

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-9 (main)
$ touch 01-memoization.js
```

7. Kemudian , Ketikkan kode tersebut ke dalam file tersebut ,

```
2  function memoize(fn) {
3      const cache = new Map();
4
5      return function (...args) {
6          const key = JSON.stringify(args);
7
8          if (cache.has(key)) {
9              return cache.get(key);
10         }
11
12         const hasil = fn.apply(this, args);
13         cache.set(key, hasil);
14
15         return hasil;
16     };
17 }
18
19 // Fibonacci
20 function fibNaif(n) {
21     if (n <= 1) return n;
22     return fibNaif(n - 1) + fibNaif(n - 2);
23 }
24
25 const fibMemo = memoize(function (n) {
26     if (n <= 1) return n;
27     return fibMemo(n - 1) + fibMemo(n - 2);
28 });
29
30 function fibIteratif(n) {
31     if (n <= 1) return n;
32
33     let a = 0;
34     let b = 1;
35
36     for (let i = 2; i <= n; i++) {
37         [a, b] = [b, a + b];
38     }
```

8. Simpan file , lalu jalankan file nya

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-9 (main)
$ node 01-memoization.js
=== Fibonacci Benchmark ===
Naif O(2^n): 1209 ms
Memo O(n): 1 ms
Iteratif O(n): 0 ms
Hasil fib(38): 39088169
```

9. Kemudian , Buatkan file baru Bernama 02-merge-sort.js di dalam folder praktikum-9 ,

```
USER@DESKTOP-HFQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-9 (main)
$ touch 02-merge-sort.js
```

10. Kemudian , Ketik Kode tersebut ke dalam File yang sudah kamu buat tersebut ,

```
1  function merge(kiri, kanan) {
2      const hasil = [];
3      let i = 0,
4          j = 0;
5
6      while (i < kiri.length && j < kanan.length) {
7          if (kiri[i] <= kanan[j]) hasil.push(kiri[i++]);
8          else hasil.push(kanan[j++]);
9      }
10
11     return [...hasil, ...kiri.slice(i), ...kanan.slice(j)];
12 }
13
14 function mergeSort(arr) {
15     if (arr.length <= 1) return arr;
16     const tengah = Math.floor(arr.length / 2);
17     const kiri = mergeSort(arr.slice(0, tengah));
18     const kanan = mergeSort(arr.slice(tengah));
19     return merge(kiri, kanan);
20 }
21
22 let depth = 0;
23 function mergeSortViz(arr) {
24     const pad = " ".repeat(depth);
25     console.log(`${pad}DIVIDE: [${arr}]`);
26     if (arr.length <= 1) return arr;
27     const mid = Math.floor(arr.length / 2);
28     depth++;
29     const L = mergeSortViz(arr.slice(0, mid));
30     const R = mergeSortViz(arr.slice(mid));
31     depth--;
32     const merged = merge(L, R);
33     console.log(`${pad}MERGE : [${L}] + [${R}] = [${merged}]`);
34     return merged;
35 }
```

11. Simpan File tersebut , Lalu jalan kan di terminal ,

```
USER@DESKTOP-HFQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-9 (main)
$ node 02-merge-sort.js
=== Visualisasi Merge Sort ===
DIVIDE: [38,27,43,3,9]
DIVIDE: [38,27]
DIVIDE: [38]
DIVIDE: [27]
MERGE : [38] + [27] = [27,38]
DIVIDE: [43,3,9]
DIVIDE: [43]
DIVIDE: [3,9]
DIVIDE: [3]
DIVIDE: [9]
MERGE : [3] + [9] = [3,9]
MERGE : [43] + [3,9] = [3,9,43]
MERGE : [27,38] + [3,9,43] = [3,9,27,38,43]

=== Benchmark Merge Sort ===
mergeSort(100000 elemen): 151 ms
Terurut? true
```

12. Selanjutnya , Buatkan file baru Bernama 03-Quick-sort.js Di dalam folder Praktikum-9 ,

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-9 (main)
$ touch 03-quick-sort.js
```

13. Berikut nya , Ketik kode ini ke dalam file 03-Quick-sort.js ,

```
1  function quickSort(arr) {
2      if (arr.length <= 1) return arr;
3      const pivotIdx = Math.floor(arr.length / 2);
4      const pivot = arr[pivotIdx];
5      const kiri = arr.filter((x, i) => i !== pivotIdx && x <= pivot);
6      const kanan = arr.filter((x, i) => i !== pivotIdx && x > pivot);
7      return [...quickSort(kiri), pivot, ...quickSort(kanan)];
8  }
9
10 function partisi(arr, lo, hi) {
11     const pivot = arr[hi];
12     let i = lo - 1;
13     for (let j = lo; j < hi; j++) {
14         if (arr[j] <= pivot) {
15             i++;
16             [arr[i], arr[j]] = [arr[j], arr[i]];
17         }
18     }
19     [arr[i + 1], arr[hi]] = [arr[hi], arr[i + 1]];
20     return i + 1;
21 }
22 function quickSortInPlace(arr, lo = 0, hi = arr.length - 1) {
23     if (lo < hi) {
24         const p = partisi(arr, lo, hi);
25         quickSortInPlace(arr, lo, p - 1);
26         quickSortInPlace(arr, p + 1, hi);
27     }
28     return arr;
29 }
```

14. Simpan file nya , Lalu jalankan di terminal untuk melihat hasil nya ,

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-9 (main)
$ node 03-quick-sort.js
=== Quick Sort Demonstrasi ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== Benchmark ===
quickSort (100000): 123 ms
quickSortInPlace(100000): 43 ms
Array.sort (builtin)(100000): 58 ms

=== Worst Case Quick Sort ===
(Dilewati untuk menghindari stack overflow pada n=10000)
Ini menunjukkan pentingnya strategi pemilihan pivot!
```

15. Berikut nya , Buatkan file baru Bernama 04-Sorting-n2.js di dalam folder praktikum-9 ,

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-9 (main)
$ touch 04-sorting-n2.js
```

16. Kemudian , Ketik kode ini di dalam file 04-Sorting-n2.js ,

```
1 // Bubble Sort
2 function bubbleSort(arr) {
3   const a = [...arr];
4   for (let i = 0; i < a.length - 1; i++) {
5     let sudahTerurut = true;
6     for (let j = 0; j < a.length - i - 1; j++) {
7       if (a[j] > a[j + 1]) {
8         [a[j], a[j + 1]] = [a[j + 1], a[j]];
9         sudahTerurut = false;
10      }
11    }
12    if (sudahTerurut) break;
13  }
14  return a;
15 }
16
17 function selectionSort(arr) {
18   const a = [...arr];
19   for (let i = 0; i < a.length - 1; i++) {
20     let idxMin = i;
21     for (let j = i + 1; j < a.length; j++) if (a[j] < a[idxMin]) idxMin = j;
22     if (idxMin !== i) [a[i], a[idxMin]] = [a[idxMin], a[i]];
23   }
24   return a;
25 }
26
27 function insertionSort(arr) {
28   const a = [...arr];
29   for (let i = 1; i < a.length; i++) {
30     const kunci = a[i];
31     let j = i - 1;
32     while (j >= 0 && a[j] > kunci) {
33       a[j + 1] = a[j];
34       j--;
35     }
36     a[j + 1] = kunci;
37   }
38 }
```

17. Setelah itu , Simpan file nya lalu jalankan di terminal untuk melihat hasil nya ,

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-9 (main)
$ node 04-sorting-n2.js
Input : [
  64, 34, 25, 12,
  22, 11, 90
]
Bubble Sort : [
  11, 12, 22, 25,
  34, 64, 90
]
Selection Sort: [
  11, 12, 22, 25,
  34, 64, 90
]
Insertion Sort: [
  11, 12, 22, 25,
  34, 64, 90
]

=== Benchmark n=5000 ===
bubbleSort: 98 ms
selectionSort: 44 ms
insertionSort: 30 ms

=== Insertion Sort pada data hampir terurut (n=5000) ===
Insertion Sort: 1 ms (jauh lebih cepat!!)
```

18. Berikut nya , Buatkan File baru di dalam folder praktikum-9 bernama 05-Counting-sort.js ,

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-9 (main)
$ touch 05-counting-sort.js
```

19. Kemudian , Masuk ke dalam file nya , Ketiklah kode ini ,

```
1 function countingSort(arr, maxVal) {
2   const count = new Array(maxVal + 1).fill(0);
3   const output = new Array(arr.length);
4
5   for (const x of arr) count[x]++;
6
7   for (let i = 1; i <= maxVal; i++) count[i] += count[i - 1];
8
9   for (let i = arr.length - 1; i >= 0; i--) {
10    output[--count[arr[i]]] = arr[i];
11  }
12  return output;
13 }
14 const nilai = [4, 2, 2, 8, 3, 3, 1, 7, 3, 5];
15 console.log("Input :", nilai);
16 console.log("Sorted :", countingSort(nilai, 9));
17
18 const N = 1_000_000;
19 const data = Array.from({ length: N }, () => Math.floor(Math.random() * 1000));
20 console.log("\n=== Benchmark n=1,000,000 ===");
21 let t = Date.now();
22 countingSort([...data], 999);
23 console.log("Counting Sort O(n) :", Date.now() - t, "ms");
24 t = Date.now();
25 [...data].sort((a, b) => a - b);
26 console.log("Array.sort (Timsort) :", Date.now() - t, "ms");
27
```

20. Simpan file tersebut , Lalu jalankan di terminal untuk melihat hasil nya ,

```
USER@DESKTOP-HFQQPOG MINGW64 ~/2025573010015_Algoritma_Dan_Stuktur_Data/praktikum-9 (main)
$ node 05-counting-sort.js
Input : [
  4, 2, 2, 8, 3,
  3, 1, 7, 3, 5
]
Sorted : [
  1, 2, 2, 3, 3,
  3, 4, 5, 7, 8
]

=== Benchmark n=1,000,000 ===
Counting Sort O(n) : 73 ms
Array.sort (Timsort) : 503 ms
```

BAB IV PENUTUP

4.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa setiap algoritma memiliki cara kerja, kelebihan, dan tingkat efisiensi yang berbeda dalam menyelesaikan proses pengolahan data. Teknik Memoization terbukti mampu meningkatkan performa program dengan menyimpan hasil perhitungan yang telah dilakukan sebelumnya sehingga dapat mengurangi perhitungan yang berulang dan mempercepat waktu eksekusi program. Selain itu, algoritma Merge Sort dan Quick Sort merupakan algoritma pengurutan yang efisien karena memiliki kompleksitas waktu rata-rata $O(n \log n)$. Merge Sort bekerja dengan membagi data menjadi bagian-bagian kecil kemudian menggabungkannya kembali dalam keadaan terurut, sedangkan Quick Sort bekerja dengan memilih sebuah pivot dan membagi data berdasarkan nilai pivot tersebut. Kedua algoritma ini sangat cocok digunakan untuk mengurutkan data dalam jumlah besar. Pada praktikum ini juga dipelajari algoritma Sorting $O(n^2)$ yang meliputi Bubble Sort, Selection Sort, dan Insertion Sort. Ketiga algoritma tersebut memiliki konsep yang lebih sederhana dan mudah dipahami, namun membutuhkan waktu yang lebih lama ketika digunakan pada data berukuran besar karena memiliki kompleksitas waktu $O(n^2)$. Melalui praktikum ini, mahasiswa dapat memahami konsep dasar optimasi program dan berbagai teknik pengurutan data, serta mampu menganalisis perbedaan performa dari masing-masing algoritma. Pengetahuan ini menjadi dasar yang penting dalam pengembangan perangkat lunak dan penyelesaian permasalahan yang berkaitan dengan pengolahan data secara efisien.

DAFTAR PUSTAKA

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). Cambridge: MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java*. New York: John Wiley & Sons.

Lafore, R. (2002). *Data Structures and Algorithms in Java* (2nd ed.). Indianapolis: Sams Publishing.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Boston: Addison-Wesley.

Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). Sebastopol: O'Reilly Media.

[MDN Web Docs - JavaScript Map Object](#)

[MDN Web Docs - JavaScript Functions](#)